

Project Report On

MUDRATRACK

GST INVOICING & JOB-CARD MANAGEMENT ERP

At Enerscript

A Project Report submitted in partial fulfillment of the requirements for the
completion of the internship project.

STUDENT DETAILS:

Name: Dipak Rajendra Patil

Roll No: MS2414

Organization: Enerscript

Project Guide: Raja Kumar (Senior Software Engineer, Enerscript)

Project Period: Academic session 2025–2026

CERTIFICATE

This is to certify that **Dipak Rajendra Patil** (Roll No: **MS2414**) has successfully completed his industrial project on: **"MudraTrack: GST Invoicing & Job-Card Management ERP"**.

The project work undertaken by the candidate demonstrates a comprehensive understanding of modern full-stack engineering, scalable monorepo architectures, and regulatory compliance workflows within enterprise ERP systems. This report fulfills the academic standards and technical criteria required for the completion of the curriculum.

Head of Department

Department of Scientific
Computing, Modelling and
Simulation

Acceptance

This manuscript has been read and accepted in satisfaction of the Industrial Training Project requirement of the internship project at Enerscript during the academic session 2025 - 2026.

Examination Committee:

Sr. No.	Name	Signature
1.	_____	_____
2.	_____	_____
3.	_____	_____

Place: _____

Date: _____

Contents

1 Project Overview	4
2 Technologies Used	5
2.1 Frontend Technology Stack	5
2.2 Backend Technology Stack	5
3 System Architecture	6
4 Database Schema Design (Prisma)	7
4.1 Entity Relationship Diagram (ERD)	7
4.2 Data Models Summary	7
5 Technical Implementation & Key Features	9
5.1 Frontend Features	9
5.1.1 1. Multi-Company Switching & Cache Synchronization	9
5.1.2 2. Mobile Responsiveness & Localization	9
5.1.3 3. Wallet Interface & Razorpay Checkout Flow	9
5.2 Backend Features	9
5.2.1 1. Dynamic Volume Slab Pricing Engine	9
5.2.2 2. Secure Webhook Validation & Ledger System	10
5.2.3 3. Notifications Dispatcher (WhatsApp WABA & Email)	10
5.2.4 4. Google Gemini Ask AI Agent	10
6 Key Learnings & Project Outcomes	11
7 Conclusion	11

1 Project Overview

MudraTrack is an enterprise-grade ERP (Enterprise Resource Planning) and billing platform custom-engineered for heavy-production businesses such as print shops, digital packaging studios, flex/banner manufacturers, and offset printing facilities.

In industrial production environments, there is typically a disconnect between production operations (e.g., job tracking, design checks, printing, cutting, finishing) and billing/financial compliance. MudraTrack bridges this gap by offering a unified workflow:

1. **Job-Card Lifecycle:** Digital tracking of manufacturing jobs through discrete stages (Pending → Printing → Cutting → Binding → PickedUp/Delivered).
2. **GST-Compliant Invoicing:** Auto-generation of professional tax invoices and proforma billing with comprehensive IGST/CGST/SGST breakdowns and HSN-code mapping.
3. **Prepaid Wallet (Razorpay):** A commercial tenant-wallet model where clients or sub-organizations recharge a balance online to automatically draw down services.
4. **Automated Messaging & Webhooks:** Dispatching PDF receipts, status notifications, and invoice alerts dynamically over WhatsApp and Email.
5. **AI Business Intelligence (Gemini integration):** An integrated natural language AI chat interface using Google Gemini function calling to allow owners and team members to query operational data (e.g., revenue reports, AR totals, overdue items) securely.

2 Technologies Used

MudraTrack leverages a modern, scalable monorepo architecture managed via **Nx Build System** to maintain a clean separation between frontend and backend services:

2.1 Frontend Technology Stack

- **Core:** React.js (v18)
- **State Management:** Redux Toolkit (global UI state and local offline transitions)
- **API Client:** Apollo Client (GraphQL query and mutation layer, utilizing cache-level sync for switching corporate tenants)
- **UI System:** Material UI (MUI v5) and custom themes styled with vanilla CSS
- **Animations:** Framer Motion (for fluid dashboard page transitions, active sidebars, and status changes)
- **SDKs:** Razorpay Checkout SDK (web integration)

2.2 Backend Technology Stack

- **Framework:** NestJS (modular Node.js framework using TypeScript)
- **Query Layer:** GraphQL (Code-first schema resolver approach)
- **Database Engine:** MongoDB (highly flexible document schema ideal for embedded job item metrics and logs)
- **ORM Layer:** Prisma ORM (schema modeling, database migrations, and type-safe query generation)
- **Automation:** Node-cron (for checking invoice recurrence schedules and processing daily payment reminders)
- **AI Integration:** @google/generative-ai (Gemini model orchestration with function calling)
- **Messaging Engines:** Twilio/WhatsApp Business API (WABA) for direct chat dispatches, AWS SES for standard notification mailings, and OAuth2-based integrations for user-owned Gmail and Outlook mailboxes.

3 System Architecture

The architecture relies on a central GraphQL-based NestJS API serving a React Single Page Application (SPA). Data interactions are modeled using Prisma ORM against MongoDB. The diagram below illustrates the comprehensive system boundaries and integration points:

```
graph TD
    subgraph Frontend [Client - React SPA]
        A[React UI Pages] --> B[Redux State Manager]
        A --> C[Apollo Client Cache]
    end

    subgraph Backend [NestJS Server API]
        D[GraphQL Resolvers] --> E[NestJS Services]
        F[JWT Auth Middleware] --> D
        E --> G[Slab Pricing Engine]
        E --> H[Gemini AI Manager]
        E --> I[Cron Scheduler]
    end

    subgraph Database [Storage Layer]
        J[(MongoDB Cluster)]
    end

    subgraph Integrations [External Providers]
        K[Razorpay Gateway]
        L[Google Gemini API]
        M[WhatsApp WABA API]
        N[AWS SES / Gmail / Outlook]
    end

    C -- GraphQL Queries & Mutations --> D
    E -- Prisma ORM Client --> J
    B -- SDK Checkout --> K
    K -- SHA256 Webhooks --> E
    H -- Function Calling --x L
    E -- Send Messages --> M
    E -- SMTP/OAuth Mail --> N
```

4 Database Schema Design (Prisma)

MudraTrack's database is modeled using a Prisma schema optimized for MongoDB. The entity relationships are mapped securely below:

4.1 Entity Relationship Diagram (ERD)

```
erDiagram
    USER ||--o{ COMPANIES_USERS : "associated through"
    COMPANY ||--o{ COMPANIES_USERS : "links user to"
    USER }|--|| COMPANY : "has current active"

    COMPANY ||--o{ CUSTOMER : "manages"
    COMPANY ||--o{ INVOICE : "issues"
    COMPANY ||--o{ TRANSACTION : "records ledgers"
    COMPANY ||--o{ WALLET_RECHARGE : "records deposits"

    CUSTOMER ||--o{ JOBCARD : "places orders for"
    JOBCARD ||--o{ JOB : "contains print/cut"
    JOBCARD |o--o| INVOICE : "linked billing to"

    INVOICE ||--|{ INVOICE_ITEM : "contains items"
    INVOICE ||--o{ PAYMENT : "receives"
```

4.2 Data Models Summary

Entity	Primary Data & Fields	Purpose / Relationship
User	ID, Username, Email, Phone, Role connection, active company reference, OAuth credentials for Gmail/Outlook.	Manages credentials, authorization scopes, and custom email outbound integration per operator.
Company	ID, Name, GST Number, Wallet Balance, WABA connection tokens, E-invoice username/password.	Models the tenant company, tracks remaining prepaid funds, and holds external integration configurations.
Customer	ID, Customer Code, Billing/Shipping Address, custom service pricing slab rates.	Connects purchasers to custom rate agreements and GST invoicing parameters.
JobCard	Card number, customer connection, total amount, creation logs, jobs list.	Serves as the workshop production traveler, grouping individual operations together.
Job	Job type (Flex, POD, Sublimation, Digital), Status (Pending, Printing, Cutting, etc.), process details.	Tracks individual shop-floor processes and state machines during manufacturing.
Invoice	Invoice number, dates, items table, balance details, recurring schedule (cron data), status (Paid, Partial, Pending).	Serves as the primary financial invoice with statutory taxes (CGST, SGST, IGST).
Wallet Recharge	Razorpay Order ID, recharge amount, completion status, timestamp.	Records deposit orders generated through the online checkout portal.
Transaction	Ledger ID, transaction type (debit/credit), amount, purpose description.	Acts as the double-entry accounting journal for auditing tenant wallets.

5 Technical Implementation & Key Features

5.1 Frontend Features

5.1.1 1. Multi-Company Switching & Cache Synchronization

Commercial tenants must have separate data silos. In the React app, switching companies updates the user state. To prevent old company data from showing, Apollo Client's in-memory cache is fully reset and re-synchronized upon switching companies:

- Updates the active company reference in the global state container.
- Invokes an automatic reset on Apollo Client (`client.resetStore()`).
- Clears active cache sheets and queries background refetches for the new company workspace.

5.1.2 2. Mobile Responsiveness & Localization

- **Responsive Overhaul:** Large data tables (e.g. flex size calculations, invoice lists) stack into responsive list cards on smaller screens using CSS Flexbox/Grid and Material UI breakpoints.
- **Localization (Hindi & Marathi):** Added complete translations for local shop floor operators, allowing seamless toggling between English, Hindi, and Marathi.

5.1.3 3. Wallet Interface & Razorpay Checkout Flow

Recharging the prepaid account triggers an online payment flow. The client-side Razorpay SDK orchestration is shown in the flowchart below:

flowchart TD

```
A[User requests wallet recharge] --> B[Server generates Razorpay Order ID]
B --> C[React UI loads Razorpay Checkout SDK]
C --> D[User completes payment interface]
D --> E[Payment status check]
E -- Payment Fails --> F[Display transaction failure to User]
E -- Payment Succeeds --> G[Await Razorpay Webhook Confirmation]
```

5.2 Backend Features

5.2.1 1. Dynamic Volume Slab Pricing Engine

To automate high-volume offset/flex printing calculations, a slab-based rate discount calculator was built into the billing logic. Rather than multiplying a static unit price by quantity, the system looks up dynamic slabs:

```
[ Quantity Ordered ]
|
|---> 1 - 10 Units    --> Base Rate Applied ($10/unit)
|---> 11 - 50 Units   --> Medium Discount Applied ($9/unit)
\---> 51+ Units       --> Bulk Discount Applied ($8/unit)
```

$$\text{Total Invoice Amount} = \text{Quantity} \times \text{SlabRate}(\text{Quantity}) \quad (1)$$

5.2.2 2. Secure Webhook Validation & Ledger System

To protect against payment spoofing, the Razorpay webhook listener verifies signatures using SHA256 HMAC encryption:

sequenceDiagram

autonumber

participant Razorpay as Razorpay API Gateway

participant Server as NestJS Backend Server

participant DB as MongoDB (Prisma)

Razorpay->>Server: HTTP POST Webhook (Event: payment.captured, Signature)

Note over Server: Compute SHA256 HMAC of body
using webhook secret

Server->>Server: Compare computed signature with header

alt Signatures Match

Server->>DB: Execute atomic database transaction

Note over DB: 1. Increment Company.walletBalance
2. Create Transaction Ledger entry

DB-->>Server: Transaction Committed

Server-->>Razorpay: HTTP 200 OK

else Signatures Mismatch

Server-->>Razorpay: HTTP 400 Bad Request

end

5.2.3 3. Notifications Dispatcher (WhatsApp WABA & Email)

- **WhatsApp Dispatcher:** Calls WhatsApp Business Cloud API to send pre-approved templates (order confirmations, delivery status updates, and links to PDF receipts).
- **Email Engine:** Routes transactional emails through standard AWS SES or, if the company owner connects their Gmail/Outlook accounts via OAuth2, sends them directly from their business addresses.

5.2.4 4. Google Gemini Ask AI Agent

Integrates gemini-1.5-pro with functional tools, allowing users to query business statistics:

flowchart TD

A[User enters query: 'Show overdue invoices'] --> B[Send request to Gemini API]

B --> C{Gemini decides tool call is needed?}

C -- No --> D[Return text answer directly]

C -- Yes --> E[Execute local handler: run_company_report]

E --> F[Query Database via Prisma Client]

F --> G[Pass database results back to Gemini]

G --> H[Gemini generates natural language response]

H --> I[Display answer to User in chat UI]

6 Key Learnings & Project Outcomes

1. **Monorepo Development:** Gained experience managing large codebases using Nx monorepos, optimizing build targets, and configuring caching.
2. **Financial Cryptography:** Implemented payment verification using secure webhook signatures.
3. **Localization Principles:** Managed complex app localization across multiple languages.
4. **AI Orchestration:** Designed and integrated AI agents with database access using Gemini function calling.

7 Conclusion

MudraTrack bridges the gap between workshop operations and enterprise billing. By combining production tracking with a secure prepaid wallet, automated messaging, and an AI assistant, it offers a modern ERP solution tailored to printing and manufacturing businesses. The project successfully demonstrates the integration of complex technologies into a cohesive, production-ready application.